



FOR JAVA DEVELOPERS MOVING TO NESTJS

Customer left. Card charged.

Spring Boot to NestJS, episode 34: RxJS for Spring developers

RxJS and Reactor share operators, not runtime contracts.

A client left half a second into a two second charge. WebFlux cancelled and charged nothing; our Nest on Express route charged.

A handler returning 1, 2, 3: WebFlux answered [1,2,3], Nest answered 3.

A thousand tasks with no concurrency argument: Reactor's flatMap ran about 256 at once, RxJS mergeMap all 1000.

In scope

- Mono and Flux to Observable
- flatMap and mergeMap
- Returning a stream from a handler
- Cancellation on disconnect

Not in scope

- Schedulers
- Backpressure strategies

Measured on Nest with Express, the default HTTP adapter.

01 The measurements

HANDLER RETURNS 1, 2, 3	WebFlux [1,2,3]; Nest 3
flatMap, NO ARGUMENT	about 256 in flight
mergeMap, NO ARGUMENT	1000 in flight
mergeMap(work, 256)	256 in flight
CLIENT LEAVES AT 500 ms	WebFlux 0 charged; Nest 1
NEST WITH takeUntil	0 charged

Both servers real processes.

02 Wire the disconnect into the Observable

Our Nest on Express route did not cancel the returned Observable when the client left. Stop it when the response closes unfinished, and keep the side effect idempotent.

```
payments.controller.ts

@Get('charge')
charge(@Res({ passthrough: true }) res: Response): Observable<string> {
  const clientLeft = fromEvent(res, 'close').pipe(filter(() => !res.writableFinished));
  return this.payments.charge().pipe(takeUntil(clientLeft));
}
```

with takeUntil: charged afterwards 0

Run on a real machine, 2026-10-01, by scripts/verify-rxjs.sh in the course repository.

03 Give mergeMap a limit

Reactor's flatMap defaults to about 256 inner subscriptions; RxJS mergeMap has none. Pass the second argument.

```
fan-out.ts

from(ids).pipe(mergeMap((id) => this.http.get(url(id)), 8));
```

mergeMap with 256: peak in flight 256

Run on a real machine, 2026-10-01, by scripts/verify-rxjs.sh in the course repository.

Before you trust an Observable in Nest

1. Does the handler need every value, or the last one?
2. Does every mergeMap that calls out have a concurrency limit?
3. What happens to the work when the client disconnects?
4. Is the side effect idempotent?
5. Is the Observable cold, and does anything subscribe twice?



Everything else

- | | |
|--|---|
|  Site
https://code-with-sam-dev.github.io |  Facebook
https://www.facebook.com/codewithsamdev |
|  YouTube
https://www.youtube.com/@CodewithSam-Dev |  GitHub
https://github.com/code-with-sam-dev |
|  LinkedIn
https://www.linkedin.com/company/code-with-sam-dev |  Buy me a coffee
https://buymeacoffee.com/codewithsam |
|  x
https://x.com/CodeWithSamDev |  Buy me sadza
https://buymesadza.com/codewithsam |
|  TikTok
https://www.tiktok.com/@codewithsamdev | Every measurement, one command: scripts/verify-rxjs.sh
https://github.com/code-with-sam-dev/spring-to-node |
|  Instagram
https://www.instagram.com/codewithsamdev | RxJS operators
https://rxjs.dev/guide/operators |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.